Breaking the Ratchet - Technical Report

Breaking the Ratchet — Technical Report

Names / roll numbers: *TODO: fill in* **Submission:** `as3_rollno1_rollno2/`

> **Note on filename:** rename this file's roll numbers and the containing directory once you have your real roll number(s), then convert to PDF (`pandoc report_rollno1_rollno2.md -o report_rollno1_rollno2.pdf`, or paste into Word/Google Docs and export) before submitting, per the handout's required deliverable name `report_rollno1_rollno2.pdf`.
>
> Sections marked **`[FILL IN FROM YOUR RUN]`** need real evidence from your own `docker compose` lab session (console logs, `docker stats` numbers/plots, packet-capture excerpts) — this draft could not produce those because this working environment has no Docker daemon available. Everything else (vulnerability analysis, code behavior, pseudocode fixes) is derived directly from `protocol.md` and was verified by building and running the attack PoCs against a standalone simulated server/client (see `attacks/README.md` and the Generative-AI appendix).

---

1. Background

The lab implements X3DH (protocol.md §3) + Double Ratchet (protocol.md §4) over a JSON-line TCP protocol (protocol.md §2), with ChaCha20-Poly1305 AEAD, X25519 DH, Ed25519 signatures, and HKDF-SHA256/HMAC-SHA256 KDFs. Delivery is pull-based through an in-memory relay server: `send` enqueues a `Frame` in the recipient's mailbox, `poll` drains it.

Three implementation-level weaknesses are examined below, matching Attacks 0–2 of the assignment.

---

2. Attack 0 — MITM message-counter manipulation / resource exhaustion

2.A Message processing analysis

The message number and previous-message-number fields live in the `Frame` header (protocol.md §2.3): `header.n` (this message's position in the sender's current symmetric chain) and `header.pn` (the length of the sender's *previous* sending chain, used so the receiver knows how many trailing messages on the old chain it may still need to accept). Both are plain JSON integers, not separately signed or MAC'd.

Whether tampering with them "works" depends on where in `ratchet_decrypt` (protocol.md §4.4) the damage is done relative to the authenticity check:

```
        def ratchet_decrypt(st, header, ct):
            mk = try_skipped_keys(st, header)
            if mk is not None:
                return aead_decrypt(mk, ct, st.AD + serialize(header))

            if header["dh"] != st.DHr:
                skip_message_keys(st, header["pn"])
                dh_ratchet(st, header)
            skip_message_keys(st, header["n"])          # <-- (1) runs
unconditionally

            mk, st.CKr = kdf_ck(st.CKr)
            st.Nr += 1
            return aead_decrypt(mk, ct, st.AD + serialize(header))   #
<-- (2) auth check
```

The AEAD associated data is `st.AD + serialize(header)` — the header *is* cryptographically bound to the ciphertext. So a header modified in transit (different `n`/`pn`) changes the AD the receiver recomputes at step (2), which no longer matches the AD the real sender used to produce the authentication tag. **The forged frame is therefore expected to fail authentication and the plaintext is never recovered or accepted as genuine.** This matches the "if the header is authenticated and cannot be modified successfully" branch the assignment anticipates.

However — and this is the actual implementation flaw — **step (1) runs before step (2)**. `skip_message_keys` walks the receiving chain with `kdf_ck` (two HMAC-SHA256 calls per step) from the current `Nr` up to the attacker-supplied `header["n"]`, storing one entry per skipped key in `st.skipped`, *before any signature/MAC check on the attacker-controlled n has happened*. The only bound is `MAX_SKIP = 1000` (`if until - st.Nr > MAX_SKIP: raise Error(...)`), so a single tampered frame can force the receiver to perform up to 1000 KDF steps and grow its in-memory skipped-key table by up to 1000 entries — all of which has already happened by the time `aead_decrypt` rejects the frame. Repeating this (a fresh tampered frame on every delivery) lets an attacker sitting between the server and a victim burn CPU and grow the victim's memory footprint indefinitely, **without ever needing a valid key or producing a single decryptable message.** If the same attack is mounted via the `header["dh"] != st.DHr` branch (a

forged new DH public key), the victim additionally performs a wasted `dh_ratchet()` — generating a fresh X25519 keypair and two extra `kdf_rk` HKDF calls — per tampered frame.

This is the classic "commit work before verifying" ordering bug: resource expenditure and authentication are not atomic.

2.B Attack implementation

`attacks/attack0.cpp` implements Mallory as a transparent TCP proxy between the real server and Bob (per the assignment's scenario diagram). It passes every byte through unmodified **except** `"frames"` responses to Bob's `poll` requests: for each `Frame` addressed to Bob, it parses `header.n` (and optionally `header.pn`) and rewrites it to an attacker-chosen gap before forwarding. No cryptographic material is needed for this attack — Mallory never has to decrypt or forge a valid ciphertext, only edit an integer in a JSON object sitting in plaintext on the wire, which is possible precisely because the transport itself carries no TLS/transport-level integrity (protocol.md §2.1: "the transport is TCP", with no additional wrapper).

Validated locally with a scripted fake server/client pair (see `attacks/README.md`): a frame with `header.n: 1` is observed to arrive at the victim with `header.n: 999` after passing through the proxy, logged as:

```
[mallory] tampered frame to=bob from=alice : n 1 -> 999
```

Assumptions this attack depends on: 1. Mallory can occupy a MITM position between the server and Bob (in the lab, trivially arranged by pointing Bob at Mallory's address with `--serve-addr`/`--port`, modeling what ARP spoofing or a compromised router would achieve on a real network). 2. The transport provides no integrity protection of its own (plain TCP, no TLS) — confirmed by protocol.md §2.1. 3. The header fields are not protected by anything *other* than the per-message AEAD tag — i.e., nothing stops the bytes from being edited before they reach the receiver's ratchet code; only the final `aead_decrypt` call catches it, and only after the cost above has already been paid.

2.C Impact measurement `[FILL IN FROM YOUR RUN]`

Run (see `attacks/README.md` §attack0) with `docker stats bob` (or the container name shown by `docker ps`) sampled once per second while repeating sends through Mallory at increasing `--gap-n` values (suggested: 10, 100, 500, 999, 1000, 1001, and a sustained burst at 999 to show cumulative growth of `st.skipped`). Record:

- ☐ A table or plot of Bob's CPU% and memory (MiB) vs. `--gap-n`.
- ☐ A plot of Bob's memory over time while Mallory repeats the attack (e.g. one tampered frame every 2s for 60s) to show the `skipped` map growing without bound rather than being capped.
- ☐ Bob's own console log lines for a tampered frame (does it print an error, silently drop, or appear to hang?).
- ☐ The point (if any) at which Bob's process becomes unresponsive or crashes, and the `--gap-n` value / cumulative tampered-frame count at which that happened.

```
[INSERT PLOT/TABLE HERE]
```

2.D Proposed fix

Two independent fixes address the two root causes (tampering surviving authentication detection, and work being spent before verification):

```
        MAX_SKIP = 1000
        MAX_SKIP_PER_RATCHET_EPOCH = 50   # NEW: cap total skipped-key
storage growth

        def ratchet_decrypt_fixed(st, header, ct):
            mk = try_skipped_keys(st, header)
            if mk is not None:
                return aead_decrypt(mk, ct, st.AD + serialize(header))

            # --- FIX 1: bound the damage BEFORE doing any chain-
walking work ---
            # Reject absurd jumps outright, independent of whether the
frame will
            # later turn out to be authentic -- a legitimate peer
never needs to
            # skip more than a small, policy-defined number of
messages in one
            # delivery gap.
            projected_n = header["n"]
            if header["dh"] != st.DHr:
                if header["pn"] - st.Ns_of_their_chain_we_tracked >
MAX_SKIP_PER_RATCHET_EPOCH:
                    raise Error("rejecting implausible pn gap before
spending any KDF work")
            elif projected_n - st.Nr > MAX_SKIP_PER_RATCHET_EPOCH:
                raise Error("rejecting implausible n gap before
spending any KDF work")

            # --- FIX 2: verify-then-commit. Try to derive the
*candidate* message
            # key and tentatively authenticate BEFORE mutating
st.Nr/st.CKr and
            # BEFORE writing anything into st.skipped. Only commit
state changes
            # if authentication succeeds. ---
            trial_st = clone_ratchet_state(st)   # cheap: just the
small scalar/chain fields
            if header["dh"] != st.DHr:
                skip_message_keys(trial_st, header["pn"])
                dh_ratchet(trial_st, header)
            skip_message_keys(trial_st, header["n"])
            mk, trial_ckr = kdf_ck(trial_st.CKr)

            plaintext = aead_decrypt(mk, ct, st.AD +
serialize(header))  # raises/returns None on failure
            if plaintext is None:
```

```
                # Nothing about st was ever mutated -- no wasted
memory, no stuck
                # ratchet state, attacker gains nothing from forged
counters.
                raise Error("authentication failed; frame discarded,
no state changed")

            # Only now commit the trial state as the real one.
            st.__dict__.update(trial_st.__dict__)
            st.CKr = trial_ckr
            st.Nr += 1
            return plaintext
```

Key ideas, either of which alone would materially help, and which the real implementation should do together:

- **Bound first, walk later**: reject implausible `n`/`pn` jumps with a tight, application-appropriate cap (tens, not `MAX_SKIP`'s hundreds) before doing any KDF work, independent of whether the message will later authenticate.
- **Verify-then-commit**: compute the candidate message key and attempt AEAD decryption *before* mutating the receiver's persistent ratchet state (`Nr`, `CKr`, `skipped`) or performing a DH ratchet. Only commit those side effects once the AEAD tag has verified. This makes a forged frame's cost O(1) regardless of the attacker's chosen gap, because the expensive/stateful work only happens for frames that were already going to be accepted.
- Additionally cap the *total* size of `st.skipped` (evict oldest entries, or refuse once a ceiling is hit) so that even a stream of distinct, individually-small, legitimately-skipped gaps can't be abused to grow memory unboundedly over many messages.

---

3. Attack 1 — Prekey Pogo (one-time prekey exhaustion)

3.A Prekey lifecycle analysis

Per protocol.md §5 ("One-time prekey (OPK) lifecycle"):

- Prekeys are generated by the client and uploaded via `publish` (identity key + signed prekey + a batch of OPKs; `OPK_COUNT=5` by default per `docker-compose.yml`). Clients "replenish by publishing again."
- **Fetching a bundle *consumes* an OPK**: "Each `fetch_bundle` hands out — and consumes — one of the target's one-time prekeys; the `count` field reports how many remain." Consumption therefore happens at *fetch* time, not when the fetched material is actually used to complete a handshake (i.e. not when the `init` block referencing that `opk_id` is actually delivered and processed by the real recipient).
- The spec does not describe any binding between "who is allowed to call `fetch_bundle` on a given victim" and "who is actually about to message them," nor any rate limit on how often one client may call `fetch_bundle`. Any registered user — including one with no prior relationship to the victim — may call it as many times as they like.

- “Once the inventory is empty the server refuses `fetch_bundle`, so **no new session can be started** with that user until they publish more prekeys.” This is a hard availability cliff, not a graceful degradation: once drained, X3DH initiation with that user is blocked entirely (existing sessions are unaffected, since OPKs are a handshake-time resource, but nobody who doesn’t already have a session can start one).

3.B Attack implementation

`attacks/attack1.cpp` plays Mallory exactly as the scenario specifies: an “ordinary client with no special network position.” It `registers` under an arbitrary username, then calls `fetch_bundle(bob)` in a tight loop, logging the `opk_id` consumed and the `count` remaining each time, until the server responds with an empty/absent OPK (`opk == ""`, `opk_id == 0`) or an outright error. It never sends Bob a single message and never completes a handshake — the drain happens purely from the act of fetching.

Validated locally against a scripted fake server returning a decrementing `count` (see `attacks/README.md`): attack1 correctly drains a 3-OPK inventory in 3 fetches and recognizes the server’s subsequent refusal.

`[FILL IN FROM YOUR RUN]` — against the real lab: - [ ] Console transcript of `attack1` draining Bob’s real `OPK_COUNT=5` inventory (or whatever you set via the environment variable). - [ ] Alice’s own `fetch_bundle(bob)` attempt immediately afterward, showing she can no longer start a *new* session with Bob (screenshot or copied console output of her client’s error/behavior). - [ ] Packet capture excerpt showing the empty/absent `opk`/`opk_id` field once exhausted.

`[INSERT EVIDENCE HERE]`

3.C Proposed fix

The root cause is that OPK consumption is tied to an unauthenticated, unrated, “intent-free” action (merely asking for a bundle) rather than to actual, accountable use. Two complementary mitigations:

```
        # FIX A: rate-limit / deduplicate fetch_bundle per (requester,
target) pair.
        # A legitimate initiator calls fetch_bundle for a given target
at most
        # once per handshake attempt; there is no legitimate reason
for the same
        # requester to call it dozens of times in a row.
        FETCH_WINDOW_SECONDS = 60
        MAX_FETCHES_PER_WINDOW = 3

        def handle_fetch_bundle(requester, target, server_state):
            key = (requester, target)
            now = current_time()
            recent = server_state.fetch_log.get(key, [])
            recent = [t for t in recent if now - t <
FETCH_WINDOW_SECONDS]
            if len(recent) >= MAX_FETCHES_PER_WINDOW:
```

```
                return Error("rate limited: too many fetch_bundle
calls for this target")
            recent.append(now)
            server_state.fetch_log[key] = recent
            return build_bundle_response(target, server_state)


        # FIX B: don't commit the OPK as "spent" merely for being
handed out --
        # reserve it with a short expiry, and only permanently retire
it once the
        # handshake that actually used it (the first "send" carrying
that opk_id
        # in its init block) is observed. An OPK that's handed out but
never
        # claimed within the expiry window is returned to the pool.
        OPK_RESERVATION_TTL_SECONDS = 30

        def fetch_bundle(target, server_state):
            opk = pick_available_opk(target, server_state)   # skips
reserved-and-unexpired ones
            if opk is not None:
                server_state.reserved[(target, opk.id)] =
current_time() + OPK_RESERVATION_TTL_SECONDS
            return build_bundle_response(target, opk, server_state)

        def on_first_send_with_init(init_block, server_state):
            key = (init_block.to_user, init_block.opk_id)
            server_state.reserved.pop(key, None)     # now genuinely,
permanently spent
            mark_opk_retired(key, server_state)

        def reap_expired_reservations(server_state):
            now = current_time()
            for key, expiry in list(server_state.reserved.items()):
                if now > expiry:
                    del server_state.reserved[key]   # OPK goes back
into the available pool
```

Fix A alone stops the straightforward loop this PoC uses. Fix B is the deeper structural correction: it decouples “handed out” from “spent,” closing the window even against a patient attacker who fetches slowly enough to dodge a naive rate limit, or who controls multiple registered identities to fan the requests out.

---

4. Attack 2 — Malicious server / fake-key MITM

4.A Trust model analysis

1. **How Alice obtains Bob's identity key and prekey bundle**: exclusively via `fetch_bundle` to the same server that also relays all of their ciphertext (protocol.md §2.4, §5). There is no secondary channel.
2. **How identity keys are authenticated**: the signed prekey is verified against the bundle's own `ik_sig` (`ed25519_verify(bundle.ik_sig, bundle.spk, bundle.spk_sig)`, protocol.md §3). This only proves internal self-consistency of whatever bundle was handed over — that *whoever* controls `ik_sig` also produced `spk_sig`. It proves nothing about whether that `ik_sig`/`ik_dh` pair actually belongs to the real Bob. There is no certificate authority, no TOFU/pinning persisted and checked against the previous fetch, and no manual fingerprint/"safety number" comparison described anywhere in protocol.md.
3. **Whether the server can replace key material without detection**: yes — nothing in the protocol or the client flow described in protocol.md gives Alice or Bob any value to compare against except what the same (potentially malicious) server already supplied.
4. **Independent verification mechanism**: none is described. Section 5 mentions only session persistence across client restarts, not any cross-channel identity check.

This is precisely the "fake-key attack" class the assignment's optional reference paper is about: the server is both the only channel for key distribution *and* the relay for ciphertext, with no external root of trust breaking that circularity.

4.B Attack implementation and results

`attacks/attack2.cpp` fully replaces `as3_server`. It implements `register`/`publish`/`fetch_bundle`/`send`/`poll` itself and, per real victim pair, maintains:

- a forged `FakeIdentity` it hands out whenever *anyone* asks for that victim's bundle (and later reuses as the identity it presents *as* that victim when forging toward the other side);
- the victim's *real* published bundle (stored from their genuine `publish` call) — used only internally, to let the server itself run `x3dh_initiate` against the real victim when forging a new message toward them;
- one independent Double Ratchet `RatchetState` per real victim (not per direction) — i.e., exactly two sessions total for an Alice/Bob conversation, matching the Double Ratchet's bidirectional single-state design.

On every `send`, the server: decrypts with the session it holds toward the real sender (`ratchet_decrypt`), logs the recovered plaintext, and re-encrypts the same plaintext with its separate session toward the real recipient (`ratchet_encrypt`, attaching a forged `init` block the first time it needs to initiate toward that recipient), then enqueues the result for them exactly like the genuine relay would. If a frame's AEAD check fails, the server logs that and forwards the still-opaque, unmodified ciphertext rather than fabricating a result — this is the honest-negative case the assignment asks to distinguish.

End-to-end validation performed in this environment: because this working environment lacks a Docker daemon to run the real `s4ch1n/as3` binaries, `attack2` was validated against a from-scratch Python reimplementation of protocol.md's exact

X3DH/Double-Ratchet pseudocode (standing in for the real clients, using the `cryptography` library for X25519/Ed25519/ChaCha20-Poly1305/HKDF). Result (see `attacks/README.md` §attack2 and the appendix below for the harness):

- Alice's `fetch_bundle(bob)` returns an `ik_dh` **different** from Bob's real identity key (confirmed by direct comparison) — the substitution.
- The `init` block Bob later receives, claiming to be from "alice," carries an `ik_dh` **different** from Alice's real identity key — the substitution in the other direction.
- Bob, using only his own genuine long-term keys, **successfully decrypts** the forged first message and recovers the exact plaintext Alice sent ("message whose plaintext the server can actually recover").
- Bob's reply is likewise intercepted, decrypted by the server (plaintext logged), re-encrypted, and **successfully decrypted by Alice** using only her own genuine keys.
- Neither Alice's nor Bob's (simulated) client logs any error at any point — each ran a completely valid X3DH + Double Ratchet session, just with the attacker on the other end instead of each other.

```
[evil-server] alice fetched a bundle for 'bob' -- served the FORGED
identity instead of bob's real one
[evil-server] established session with REAL alice, posing as bob
[evil-server] *** RECOVERED PLAINTEXT *** alice -> bob : "hi bob, this
is alice"
[evil-server] established session with REAL bob, posing as alice
[evil-server] *** RECOVERED PLAINTEXT *** bob -> alice : "hi alice,
bob here"
```

`[FILL IN FROM YOUR RUN]` — repeat this against the **real** lab containers (`attacks/README.md` notes an important caveat: the exact byte layout of the AEAD associated data, `serialize(header)`, is not specified in protocol.md and had to be assumed; verify it against a capture before trusting these results against the real binaries, and adjust `headerJson()`/`headerAD()` in `attack2.cpp` if needed):

- ☐ Console transcripts from the real Alice and Bob clients showing an ordinary-looking conversation with no errors.
- ☐ `attack2`'s own console log showing the recovered plaintext for at least one message each direction.
- ☐ A screenshot/log line demonstrating "a message that the server merely forwards" is not what happened here — i.e., contrast with what a non-MITM relay's log would show (no plaintext visibility) if you also ran one message through the real, honest server for comparison.
- ☐ If the real binaries reject the forged frames (AD mismatch), report that explicitly, including what you had to change (or tried and could not determine) in the AD format, and whether that failure is itself evidence the implementation binds the header more specifically than protocol.md documents.

`[INSERT EVIDENCE HERE]`

If the real implementation turns out to correctly authenticate all key-establishment material in a way this PoC cannot defeat (e.g. if there is an undocumented out-of-band pinning step the client performs that protocol.md omits), report that explicitly per the assignment's instruction, and identify which property blocked it — most likely this would be some form of TOFU persistence that `fetch_bundle` responses are checked against on the client side, which protocol.md does not currently describe.

4.C Proposed fix

The root cause is pure trust-on-first-contact with **no persistence and no out-of-band check** — not a cryptographic break of X3DH/Double Ratchet itself. Two layered fixes:

```
        # FIX A: Trust-On-First-Use (TOFU) pinning, enforced CLIENT-
SIDE (never
        # trust the server to enforce this on itself).
        def fetch_and_verify_bundle(my_local_keystore, target_user,
server):
            bundle = server.fetch_bundle(target_user)
            assert ed25519_verify(bundle.ik_sig, bundle.spk,
bundle.spk_sig)

            pinned =
my_local_keystore.get_pinned_identity(target_user)
            if pinned is None:
                # First contact ever: pin it now, but surface this to
the user
                # as a "new/unverified contact" state rather than
silently trusting it.
                my_local_keystore.pin_identity(target_user,
bundle.ik_sig, bundle.ik_dh)
                ui_warn_unverified_contact(target_user)
            elif (pinned.ik_sig, pinned.ik_dh) != (bundle.ik_sig,
bundle.ik_dh):
                # The server just handed us a DIFFERENT identity than
last time.
                # This is exactly the fake-key substitution attack 2
demonstrates.
                raise SecurityAlert(
                    f"Identity key for {target_user} changed since
last session! "
                    f"Refusing to start a new session automatically --
"
                    f"this may mean {target_user} reinstalled, OR that
the "
                    f"server is attempting a man-in-the-middle
attack."
                )
            return bundle


        # FIX B: give Alice and Bob an INDEPENDENT (non-server)
channel to compare
        # fingerprints -- the standard "safety number" mitigation.
This is the
        # only way to resolve a GENUINE first-contact fake-key attack
(Fix A alone
        # cannot distinguish "real first contact" from "attacker
intercepted the
```

```
        # very first fetch_bundle").
        def safety_number(my_identity, their_identity):
            # A short, human-comparable fingerprint of both parties'
identity
            # keys, computed identically on both ends.
            material = sorted([my_identity.ik_dh,
their_identity.ik_dh])
            digest = sha256(material[0] + material[1]).digest()
            return format_as_decimal_groups(digest)   # e.g. Signal-
style 60-digit number

        def verify_contact_out_of_band(my_identity, their_identity,
shown_to_user):
            computed = safety_number(my_identity, their_identity)
            return user_confirms_match(computed, shown_to_user)  # QR
scan / manual compare
            # Only after this returns True should the UI mark the
conversation
            # "verified" -- TOFU pinning (Fix A) protects against
*silent*,
            # *repeated* substitution; this step is what protects the
very first
            # session against an always-on malicious server.
```

Assumptions of this fix: Fix A assumes the client has durable local storage that the server cannot tamper with (reasonable — it's the client's own device) and that the *first* `fetch_bundle` for a given contact is not already compromised; it protects against a server that substitutes keys *after* a genuine first contact (e.g. mid-conversation, or for a second device), which is the easier and more common real-world case. Fix B is required to close the harder "attacker was already in the middle from message one" case, and depends on Alice and Bob having *some* channel the server doesn't control (in person, a phone call, a different app) to compare the safety number — exactly how Signal's own safety-number verification works, and consistent with the assignment's hint toward "Automatic Detection of Fake Key Attacks in Secure Messaging." **Detecting a malicious server in practice**: once Fix A is in place, any later substitution attempt produces a visible, unmissable `SecurityAlert` on the next session re-establishment — the server can stay silent about it, but it cannot stop the client from noticing the key changed, because the comparison value lives on the client, not on the server.

---

5. Appendix — Generative AI usage

Tool used: Claude (Anthropic), used as a coding assistant within this session.

What it was used for: - Reading and summarizing `assignment_pdf.pdf` and `as3/protocol.md`. - Deriving the full worked state-diagram sequence (RK/CKs/CKr/MK per message) for the required 6-message exercise, by hand-executing protocol.md's pseudocode message-by-message. - Writing `attacks/common/{json_mini.hpp,netutil.hpp,crypto.hpp}`, `attacks/attack0.cpp`, `attacks/attack1.cpp`, `attacks/attack2.cpp`, the `Makefile`, and `attacks/README.md`. - Drafting this report's vulnerability analysis and pseudocode fixes.

How the output was verified (not merely accepted): - Every C++ helper and attack program was **compiled with `-Wall`** and ran cleanly with no warnings. - `common/{json_mini,netutil,crypto}.hpp` were exercised with a standalone smoke test (`/tmp/test_common.cpp`, not shipped) asserting: JSON round-trip equality; X25519 DH agreement symmetry (`DH(a.priv,b.pub)==DH(b.priv,a.pub)`); Ed25519 sign/verify correctness; HKDF-SHA256 determinism; `kdf_rk`/`kdf_ck` output shapes; and ChaCha20-Poly1305 round-trip plus **tamper-detection** (flipping a ciphertext byte causes decryption to fail, as expected for an AEAD). - `attack0` was functionally tested against a scripted fake TCP server/client pair, confirming it correctly rewrites `header.n` in a relayed `"frames"` response while passing every other field/message through unchanged. - `attack1` was functionally tested against a scripted fake server that returns a decrementing OPK `count`, confirming it drains an inventory of 3 in exactly 3 fetches and correctly recognizes the server's subsequent refusal. - `attack2` was validated with a **full end-to-end simulation**: a from-scratch, independent Python reimplementation of protocol.md's X3DH/Double-Ratchet pseudocode (using the `cryptography` library, not reusing any of the C++ code) played the roles of honest Alice and Bob against the compiled `attack2` binary over real TCP sockets. This confirmed, with concrete assertions (not just "it ran without crashing"): the bundle served for "bob" differs from Bob's real identity key; the forged "alice" `init` block differs from Alice's real identity key; Bob's own `x3dh_respond`/`ratchet_decrypt` correctly recovers Alice's exact plaintext from the server-forged frame; and Alice's own `ratchet_decrypt` correctly recovers Bob's exact plaintext reply, relayed the same way. - The AEAD-associated-data byte-format assumption (documented inline in `crypto.hpp`/`attack2.cpp` and in `attacks/README.md`) was explicitly flagged as unverified against the real `s4ch1n/as3` binary, since this environment could not run it; the README gives the exact `tshark` command and procedure to check it before trusting attack2 against the real lab. - **Not yet verified by a human**: the real-lab empirical sections marked `[FILL IN FROM YOUR RUN]` above (actual `docker stats` numbers, real packet captures, real client console transcripts) — these require Docker, which was unavailable in the environment that produced this draft, and must be collected and reviewed by the student before submission.

Prompts (paraphrased, chronological): 1. "What's left on this branch / what does `assignment_pdf.pdf` and `protocol.md` actually require?" (research/summarization) 2. Build `attacks/common/*` matching protocol.md's crypto primitives. 3. Implement `attack0.cpp`: MITM header-counter tampering per the assignment's Attack 0 scenario. 4. Implement `attack1.cpp`: prekey exhaustion per Attack 1's scenario. 5. Implement `attack2.cpp`: full malicious-server fake-key MITM per Attack 2's scenario, including validating it end-to-end. 6. Derive the full state-diagram worked example for the required 6-message sequence. 7. Draft this report's vulnerability analysis and Python-style pseudocode fixes for all three attacks.

TODO (student): add any further prompts you issued while adapting this draft (e.g. filling in the `[FILL IN FROM YOUR RUN]` sections, or asking for clarification on a specific result), and how you verified that output too.